
MERIDIAN TELECOM

Platform Engineer Take-Home Assignment

Platform Architecture and Software specifications

Mattia Vincenzi

Date: 24/09/2026

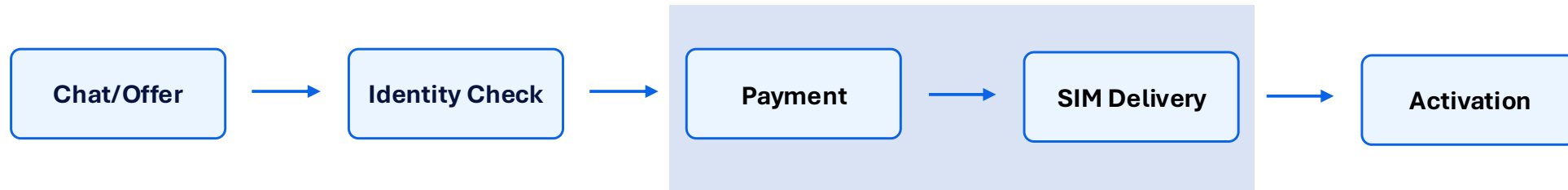
Requirements and Open Points

Platform Goals	Hard Constraints
✓ Resume conversations without losing progress or repeating actions. ✓ Support missing document uploads and manual reviews. ✓ Guarantee no duplicate charges, shipments, or premature activations.	• Team: 2 Platform Engineers (AWS/Managed Kubernetes). • Resilience: 99.9% Availability · RPO < 1 minute · RTO < 10 minutes. • Security: Confidential data must remain in the selected region. • Assumptions: Single-AZ failure scope, provider guarantees, and regional model availability at launch; Customer notification channel and provider (email/SMS/push) to be agreed.



Requirements and Open Points

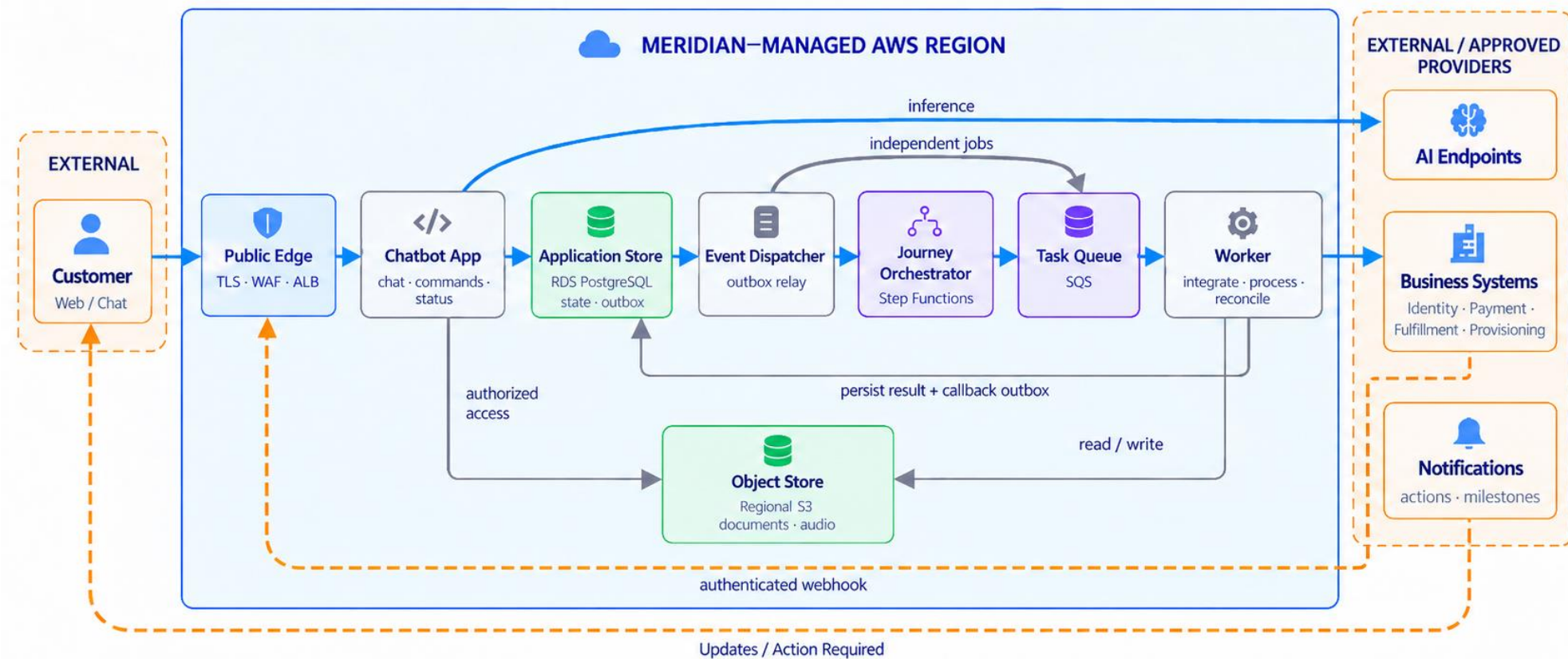
Platform Goals	Hard Constraints
✓ Resume conversations without losing progress or repeating actions. ✓ Support missing document uploads and manual reviews. ✓ Guarantee no duplicate charges, shipments, or premature activations.	• Team: 2 Platform Engineers (AWS/Managed Kubernetes). • Resilience: 99.9% Availability · RPO < 1 minute · RTO < 10 minutes. • Security: Confidential data must remain in the selected region. • Assumptions: Single-AZ failure scope, provider guarantees, and regional model availability at launch; Customer notification channel and provider (email/SMS/push) to be agreed.



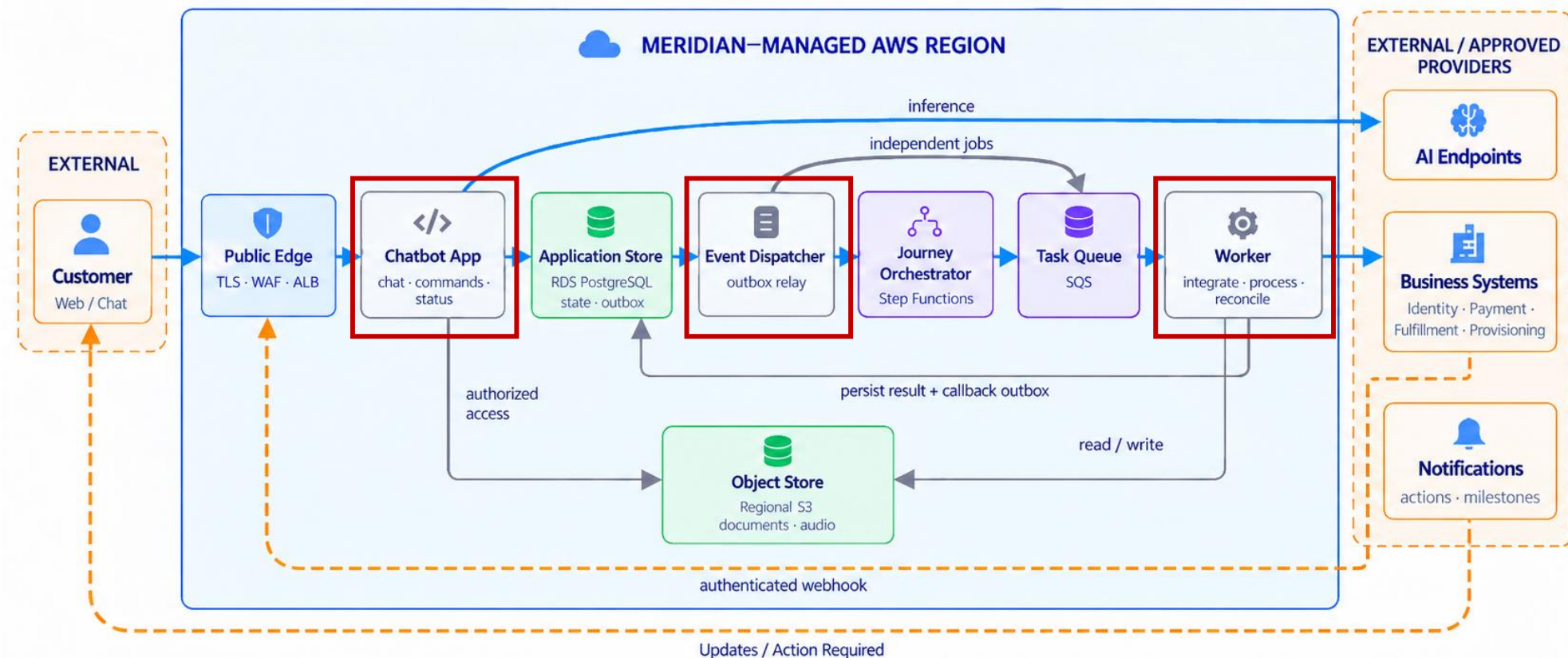
Wait State: The transition can be blocked for weeks

- Orchestration decoupled from active chat session
- Memory should be externalized from chatbot-app component

High Level Architecture: Components and Responsibilities

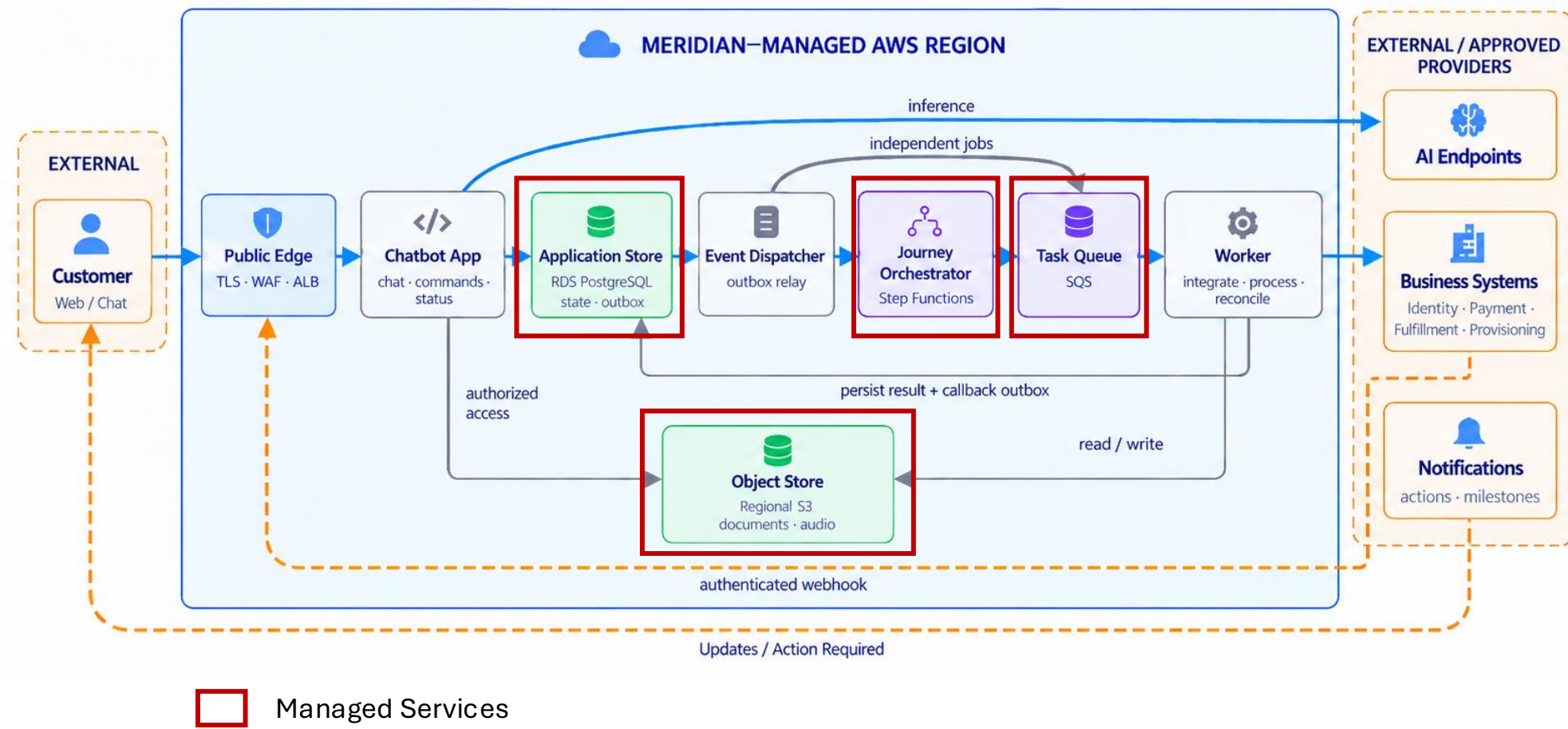


High Level Architecture: Components and Responsibilities

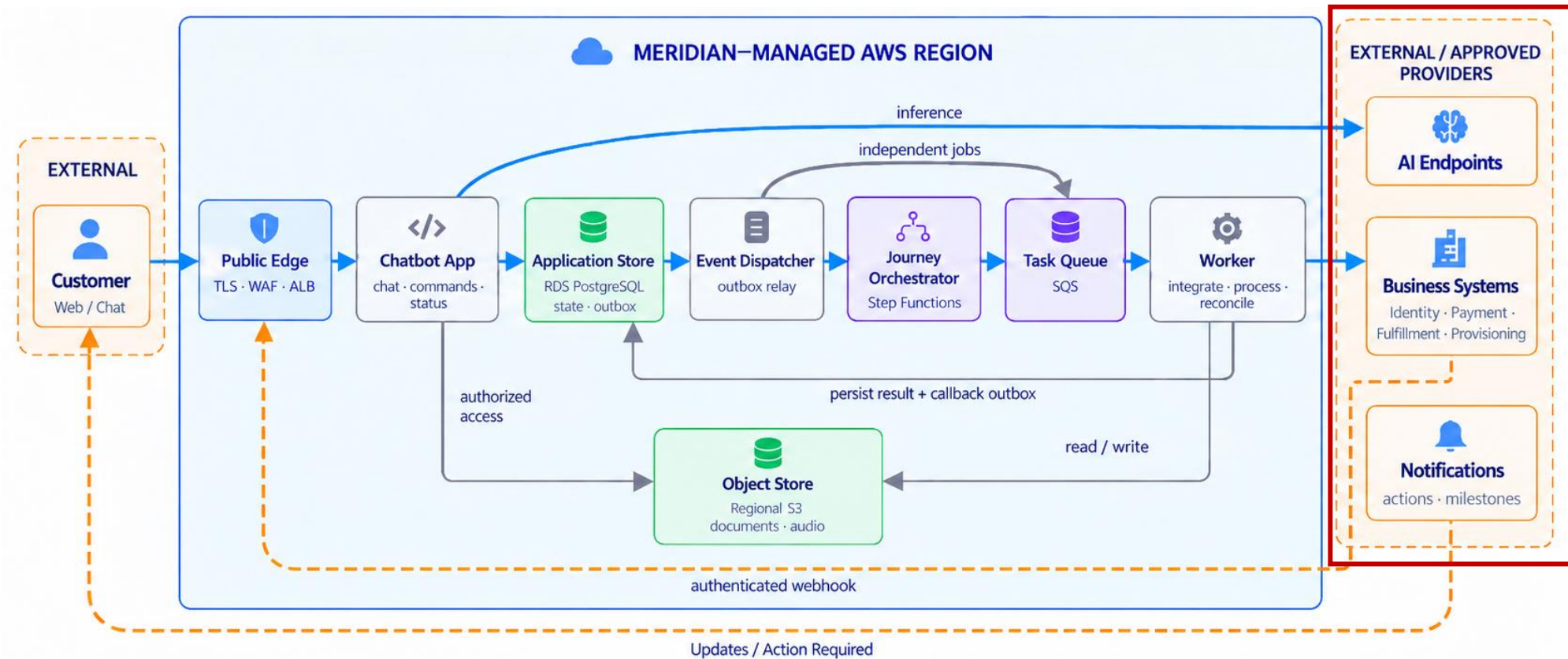


Components deployed on EKS

High Level Architecture: Components and Responsibilities



High Level Architecture: Components and Responsibilities



 External providers

Choices and trade-offs

Decision	Why	Alternatives	Trade-off
Managed orchestration - Step Functions Standard	Manage waits, timers, and resumptions with less software to maintain.	Custom workflow in PostgreSQL; Temporal.	AWS dependency, transition costs, and expressiveness limits.
Recoverable state and actions - PostgreSQL, outbox, and idempotency	Preserve progress and pending deliveries; handle duplicates with stable identities and reconciliation.	State read from workflow history; separate read store. Both require safeguards for external actions.	Polling, reconciliation logic, and provider guarantees to verify.
Spike absorption - SQS and EKS workers	Buffer slow work and scale background processing independently from chat.	Processing work inside requests; Managed queue system (RabbitMQ, NATS, etc..).	AWS dependency; Queueing delay, redelivery, and backlog control within provider quotas.
Resilient managed baseline - two AZs and RDS Multi-AZ	Reduce maintenance and failure impact for a small team.	Single-AZ to save cost; self-managed services for more control.	Higher minimum cost and AWS dependency; recovery still to demonstrate.

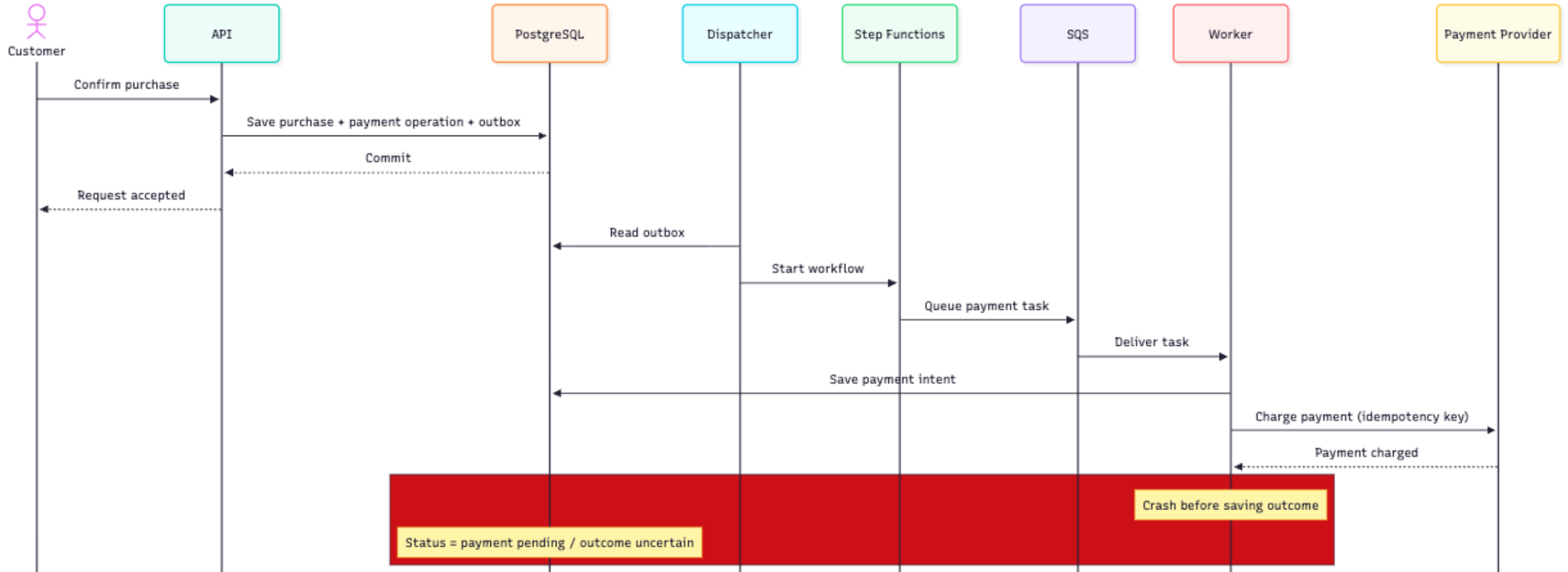
Choices and trade-offs

Decision	Why	Alternatives	Trade-off
Managed orchestration - Step Functions Standard	Manage waits, timers, and resumptions with less software to maintain.	Custom workflow in PostgreSQL; Temporal.	AWS dependency , transition costs, and expressiveness limits.
Recoverable state and actions - PostgreSQL, outbox, and idempotency	Preserve progress and pending deliveries; handle duplicates with stable identities and reconciliation.	State read from workflow history; separate read store. Both require safeguards for external actions.	Polling, reconciliation logic, and provider guarantees to verify.
Spike absorption - SQS and EKS workers	Buffer slow work and scale background processing independently from chat.	Processing work inside requests; orchestration for every independent job.	AWS dependency ; Queueing delay, redelivery, and backlog control within provider quotas.
Resilient managed baseline - two AZs and RDS Multi-AZ	Reduce maintenance and failure impact for a small team.	Single-AZ to save cost; self-managed services for more control.	Higher minimum cost and AWS dependency ; recovery still to demonstrate.

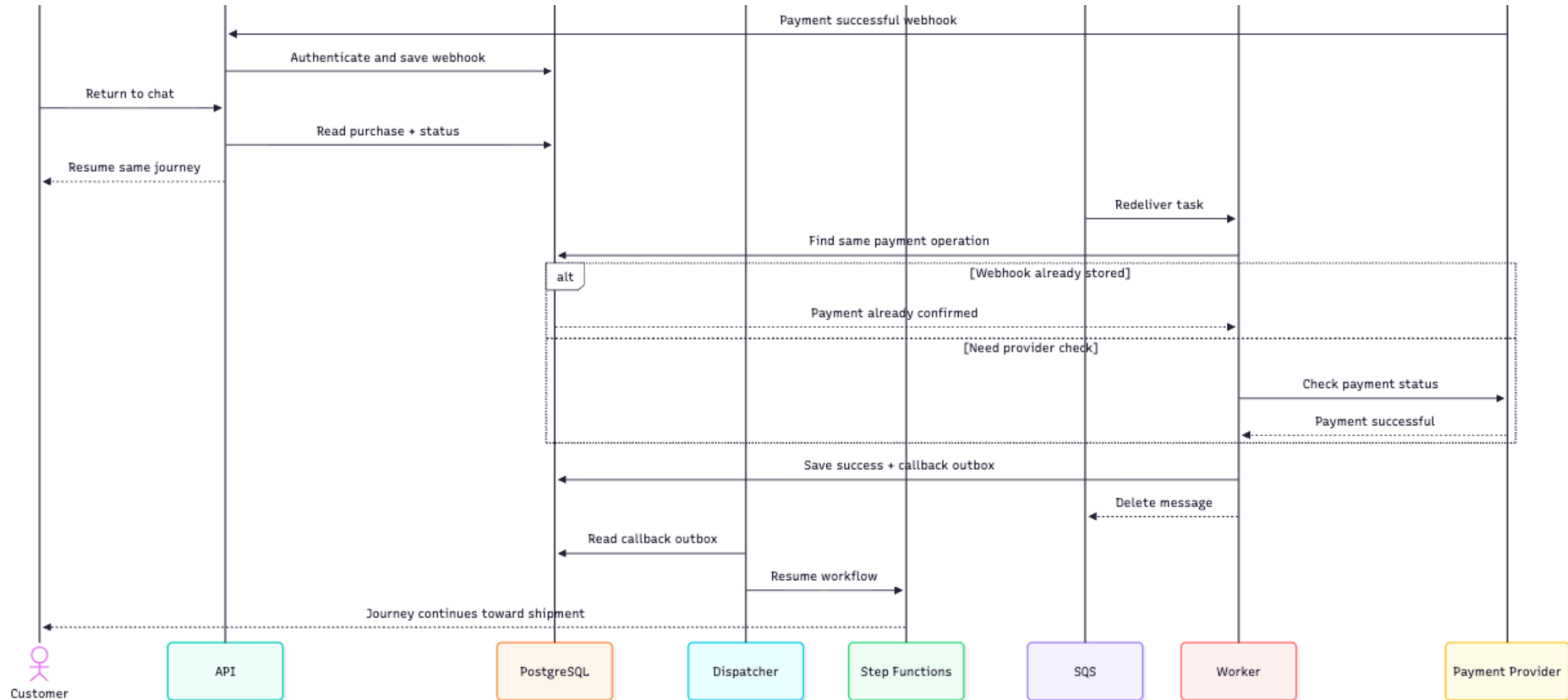
Operational focus

- For two Platform Engineers, reduce infrastructure software to maintain.

Failure example: The payment succeeds, but the worker stops (1/2)



Failure example: The payment succeeds, but the worker stops (2/2)



Resilience and recovery targets

Target	Protection	Required Validation
99.9% Availability	Replicas and node capacity distributed across two Availability Zones.	Chat and status reads survive the loss of a single zone with temporary degradation in background processing.
RPO < 1 min, RTO < 10 min	RDS Multi-AZ failover and client reconnection.	Measure the exact time required to restore usable conversations and pending work after a database primary failure.
Seamless Deployments	Versioned workflows; backward-compatible schemas and workers.	Weeks-long customer journeys successfully survive a rolling update or rollback.

Limitations

- Multi-AZ does not automatically prove targets.
- Recovery targets assume infrastructure failures within one region.
- Coverage for data corruption and regional outages remains to be agreed and validated.

Security, observability and operational responsibility

1. Protect access

- Authenticate customers, operators and provider callbacks.
- Keep workloads and databases private; expose API via TLS/WAF/ALB.
- Enforce journey ownership and least-privilege access for users and workloads.

2. Constrain AI & data

- Validate model-proposed actions in application code.
- Treat external content as untrusted and bound AI usage.
- Keep confidential data within approved regional paths.

3. Preserve auditability

- Keep unsampled audit records of agent runs, approvals and external actions.
- Correlate logs and traces by journey/operation ID.
- Redact personal data and tokens; define retention and deletion.

4. Operate daily

- **Service dashboard:** availability, latency, errors and infrastructure health.
- **Journey dashboard:** queue/outbox age, stalled journeys, uncertain outcomes and provider throttling.

Telemetry (OpenTelemetry): application logs + service/infrastructure metrics + traces → regional monitoring, dashboards and alarms

Security, observability and operational responsibility

1. Protect access

- Authenticate customers, operators and provider callbacks.
- Keep workloads and databases private; expose API via TLS/WAF/ALB.
- Enforce journey ownership and least-privilege access for users and workloads.

2. Constrain AI & data

- Validate model-proposed actions in application code.
- Treat external content as untrusted and bound AI usage.
- Keep confidential data within approved regional paths.

3. Preserve auditability

- Keep unsampled audit records of agent runs, approvals and external actions.
- Correlate logs and traces by journey/operation ID.
- Redact personal data and tokens; define retention and deletion.

4. Operate daily

- **Service dashboard:** availability, latency, errors and infrastructure health.
- **Journey dashboard:** queue/outbox age, stalled journeys, uncertain outcomes and provider throttling.

Telemetry (OpenTelemetry): application logs + service/infrastructure metrics + traces → regional monitoring, dashboards and alarms

Amazon CloudWatch (AWS-managed) vs. *Prometheus + Grafana + Loki + Tempo* (self-managed)

Scaling gradually according to external limits

≈100 launch users → path to 100,000 users

Thousands of concurrent sessions · Hundreds of thousands of daily tasks

Chat App

Growth signal:
active requests and latency

Workers

Growth signal:
backlog, job duration, and drain time

Nodes

Growth signal:
pod capacity demand

Start Simple

- Workloads share nodes, with requests/limits protecting chat from CPU-heavy processing.
- Scale workers on backlog; scale nodes via Managed Node Groups and Cluster Autoscaler.
- SQS absorbs spikes while capacity starts. HPA on app signals needs a metrics adapter.

Monitoring

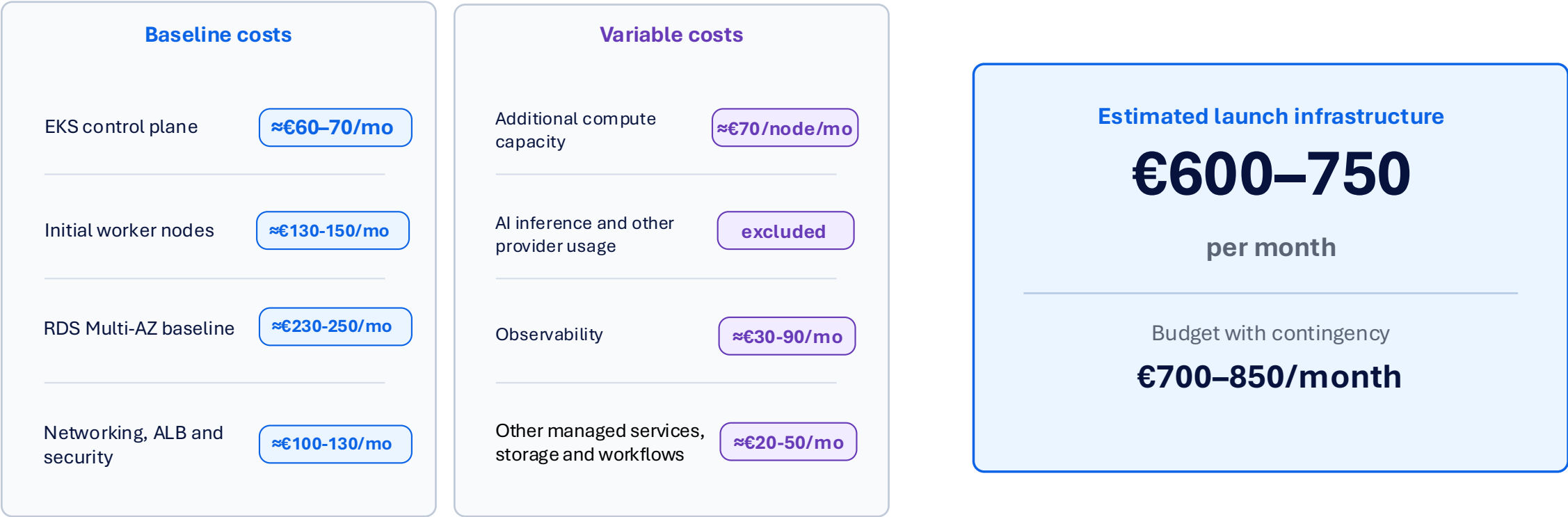
External Limits:

- DB connections
- Provider quotas
- AI budget.

If evidence confirm

- Dedicated CPU-nodes for CPU intensive workloads
- GPU devices for Meridian-managed inference models
- Caching, database access optimization etc...

Initial Cost Estimation



Sources: Ireland, September 2026. Includes baseline and assumed initial usage; excludes AI/business-provider fees and appendix items.

Delivery Roadmap Estimation

** Indicative timeline for two Platform Engineers, parallel AI-team delivery and provider sandboxes available.*



Weeks 1–2
Foundations



Weeks 3–6
Complete journey



Weeks 7-12
Launch readiness

Phase	Main activities	Expected outcome
Foundations	Confirm region, recovery scope and provider guarantees. Provision infrastructure, access controls, CI and monitoring.	Environment ready for integration; critical dependencies confirmed.
Complete journey	Integrate the end-to-end journey with AI Engineers, including reconciliation, notifications and manual-review handoff. Start failure testing.	Sandbox journey demonstrated, including resume and safe handling of retries and duplicate events.
Launch readiness	Validate load, recovery targets and deployment compatibility. Fix issues and repeated checks. Finalize test runbooks and incident ownership.	Launch approved, residency and recovery checks pass. Solution monitored with rollback procedures ready.

AI usage

- **Tools**

- **ChatGPT:** general discussion, image generation for presentation, tables, etc...
- **Codex** (GPT-5.6 Terra e GPT-6 Astra): architecture and solution design discussion, decision and trade-off evaluation. Critical review of solution against requirements.
- **NotebookLM:** image and table generation for presentation.

repo /

- **AGENTS.md**

AI working rules and review criteria: e.g.
“Treat assignment.md as the source of truth”, “Prefer the simplest credible solution for a two-engineer team”, “Ask for decisions”

- **assignment.md**

Authoritative requirements;
unchanged

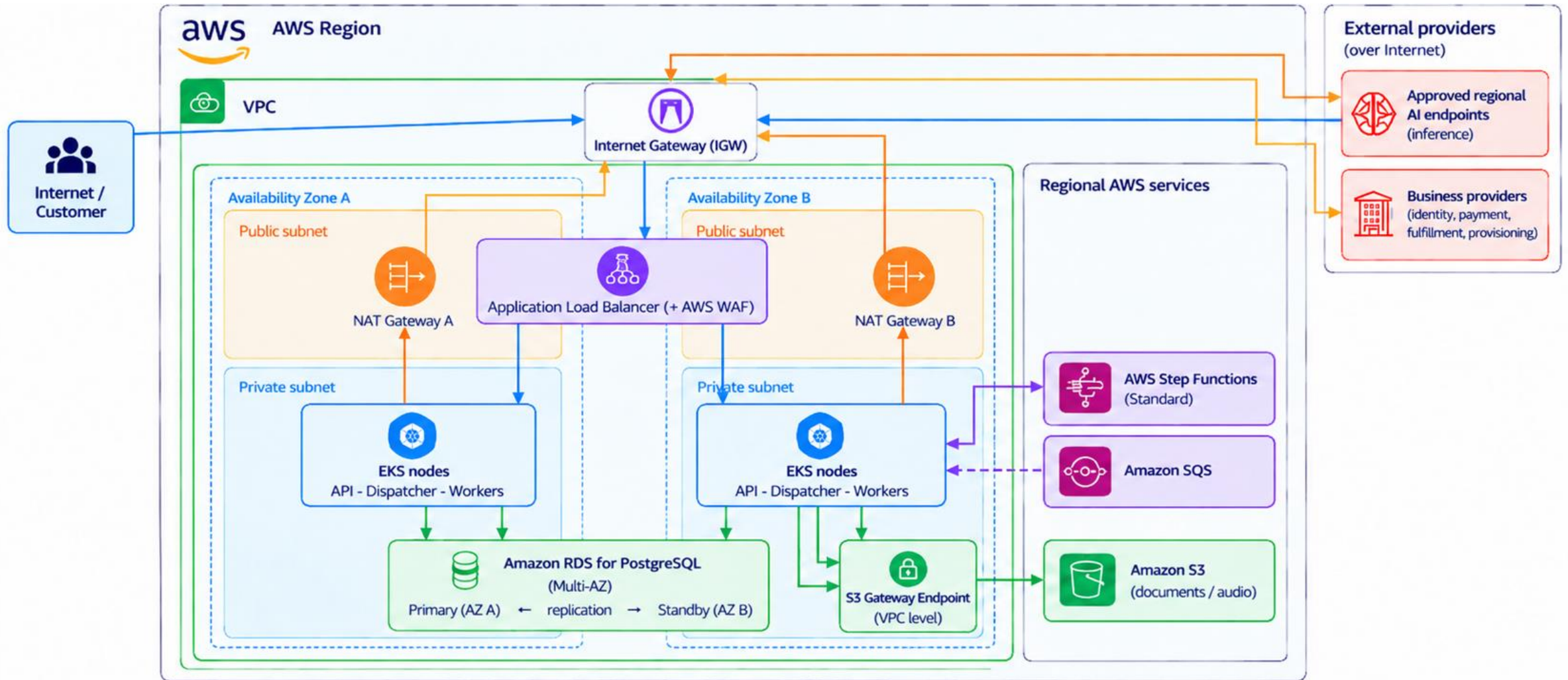
- **solution.md**

Proposal, assumptions, decisions and
validation gaps

Example of usage:

- **Accepted suggestions:** Transactional outbox pattern, managed orchestrator.
- **Delegated:** Review of alternatives, drafting and cost verification.
- **Deferred/Rejected:** GPU self-hosting since launch, Redis cache and complex custom orchestration.

Network and platform view



*Security groups restrict traffic between components.